

Fisingas

Savarankiškai talpinama fišingo atpažinimo mokymo ir vertinimo platforma Django, React ir Docker pagrindu

Tomas Vanagas

tomas.vanagas@knf.vu.lt

2026 m. rugpjūtis

Santrauka

Fisingas — savarankiškai talpinama žiniatinklio platforma fišingo laiškų atpažinimui mokyti ir vertinti, sukurta Vilniaus universiteto Kauno fakulteto auditorijoms. Studentai užsiregistruoja patys, gauna sistemos sugeneruotą prieigos kodą ir sprendžia testą: nagrinėja tikroviškas el. laiškų ekrano kopijas — su virš paveikslėlio imituojamomis nuorodų peržiūromis — ir kiekvieną laišką įvertina kaip tikrą arba fišingą, atsakydami į papildomus klausimus apie požymius. Administratoriai tvarko klausimų banką, gyvai stebi studentų progresą ir projektuoja automatiškai atsinaujinančią lyderių lentelę, kaitaliojamą su pateikties skaidrėmis. Sistema supakuota į vieną Docker Compose steką iš devynių konteinerių už vieno Caddy atvirkštinio tarpinio serverio: React vieno puslapio aplikacija, sąmoningai plonas Django JSON API, PostgreSQL duomenų bazė ir tarpinio serverio autentifikacija saugomi operatoriaus įrankiai (duomenų bazės sąsaja, failų tvarkyklės, API dokumentacija). Pagrindinis projektavimo rūpestis — *įvertinimų vientisumas kintant turiniui*: išdalijant testą kiekvieno klausimo turinys užšaldomas denormalizuotoje kopijoje studento atsakymų eilutėse, tad klausimų redagavimas ar trynimasis niekada nebegali pakeisti jau suteiktų įvertinimų. Dokumente aprašoma architektūra, duomenų modelis ir jo momentinių kopijų disciplina, autentifikacijos schema, testo gyvavimo ciklas ir tiksli vertinimo aritmetika, abiejų rolių naudotojo sąsaja bei sistemos diegimo ir eksploatavimo procedūros. Visi šio dokumento teiginiai išvesti tiesiogiai iš sistemos išeities kodo.

Turinys

1	Įvadas	3
2	Sistemos architektūra	3
2.1	Paslaugos	3
2.2	Maršrutizavimas	4
2.3	Saugumo laikysena	4
3	Duomenų modelis	5
3.1	Esybės	5
3.2	Momentinė kopija išdalijant	5
3.3	Paveikslėlių saugykla	6
4	Autentikacija ir prieigos kontrolė	6
4.1	Paskyros ir rolės	6
4.2	Sesijos	7
4.3	Autorizacija vaizduose	7
4.4	Autorizacija tarpiniame serveryje	7

5	Testo gyvavimo ciklas	7
5.1	Vienas studentas nuo pradžios iki pabaigos	7
5.2	Vertinimas	8
6	Naudotojo sąsaja	9
6.1	Technologija	9
6.2	Studento patirtis	10
6.3	Administratoriaus patirtis	10
6.4	Viešieji rodiniai	11
7	Diegimas ir eksploatavimas	11
7.1	Būtinės sąlygos	11
7.2	Steko paleidimas	11
7.3	Duomenų saugojimo terminai	12
7.4	Konfigūracijos žinynas	12
7.5	Kūrimo režimas	12
7.6	Regresiniai testai	13
8	Apribojimai ir ateities darbai	13
9	Išvados	14

1 Įvadas

Fišingas tebėra dažniausias pirminės prieigos vektorius praktikoje, o veiksmingiausia dėstytojų prieinama priemonė — treniruotė: rodyti žmonėms įtikinamas klastotes šalia tikrų laiškų ir leisti mokytis jas atskirti. Tokio pratimo vykdymas pilnai studentų auditorijai kelia reikalavimus, kuriuos bendrinės apklausų sistemos tenkina prastai:

- Medžiaga *vizuali*. Fišingo pratimas gyvena tiksliose el. pašto klientų ekrano kopijose, o esminis įgūdis — užvesti pelę ant nuorodos ir pamatyti tikrąją jos paskirtį — turi veikti ant statinio paveikslėlio.
- Registracija turi būti akimirksninė ir tinkama anonimiškumui. Studentai per paskaitą užsiregistruoja slaptyvardžiu ir gauna mašinos sugeneruotą prieigos kodą; jokių el. pašto paskyrų, jokių sąrašų importo.
- Pratimas vyksta *gyvai*. Dėstytojų reikia realaus laiko vaizdo, kas dar sprendžia, o projektuojama lyderių lentelė vertinimą paverčia žaidimu.
- Turinys nuolat kinta — klausimai pridedami, perrašomi ir išimami tarp kursų (ir jų metu) — tačiau kartą suteiktas įvertinimas privalo likti tiksliai atkuriamas.
- Platforma turi veikti savarankiškai viename universiteto serveryje ir būti prižiūrima vieno žmogaus su Docker.

Fišingas šiuos reikalavimus sprendžia sąmoningai „nuobodžiomis“ technologijomis: Django API, kuris „tik perkelia duomenis“, PostgreSQL kiekvienam būsenos baitui (įskaitant pačias ekrano kopijas), React vieno puslapio aplikacija ir Docker Compose orkestravimui. Nėra nei žinučių eilės, nei podėlio sluoksnio, nei debesijos priklausomybių. Vienintelė vieta, kur dizainas turi tvirtą nuomonę — duomenų modelis: studentui išdalytas testas yra *užšaldyta klausimų kopija*, o ne nuoroda į juos (3 skyrius).

Toliau dokumentas organizuotas taip. 2 skyriuje pristatoma konteinerių topologija, maršrutizavimas ir saugumo laikysena. 3 skyriuje aprašomas duomenų modelis ir momentinių kopijų disciplina. 4 skyriuje patikslinama autentifikacijos ir prieigos kontrolės schema. 5 skyrius seka vieną studentą per visą testą ir pateikia vertinimo aritmetiką. 6 skyriuje aprašoma naudotojo sąsaja studentams, administratoriams ir projektoriui. 7 skyrius — operatoriaus vadovas: diegimas, pagalbinės paslaugos, duomenų saugojimo terminai ir konfigūracijos žinytas. 8 skyriuje aptariami žinomi apribojimai.

2 Sistemos architektūra

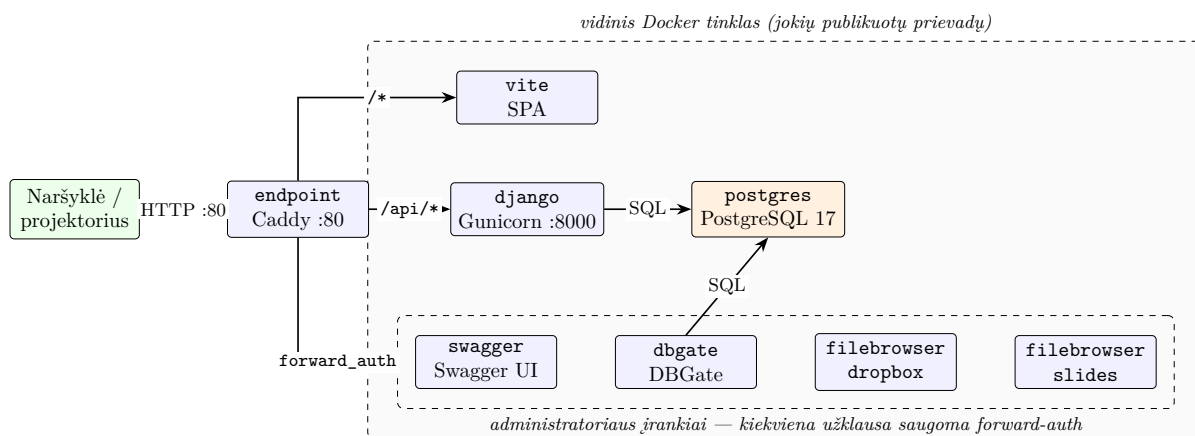
2.1 Paslaugos

Steką sudaro devyni konteineriai, deklaruoti viename `docker-compose.yml`. Jie apibendrinti 1 lentelėje ir pavaizduoti 1 paveiksle.

Django serverinė dalis sąmoningai plona: paprastos funkcijos-vaizdai be Django REST Framework, be `django.contrib.admin` ir be `django.contrib.auth` — įdiegtų aplikacijų sąrašė tėra sesijų karkasas ir trys projekto aplikacijos (`users`, `phishing_test`, `leaderboard`). Visas turinio tvarkymas vyksta per React administratoriaus sąsają; visa prieiga prie duomenų bazės eina per ORM — vienintelė „žalia“ SQL užklausa kodo bazėje yra migracija, pavertusi laiko žymų stulpelius gimtaisiais datos tipais (3 skyrius). Kiekviena užklausa vykdoma transakcijoje (`ATOMIC_REQUESTS = True`), tad pusiau įrašyta testo būsena negali išlikti.

Konteineris	Technologija	Vaidmuo
fisingas-endpoint	Caddy 2.11	Vienintelis įėjimo taškas; maršrutai, saugumas
fisingas-vite	React 19 + Vite 7, aptarnauja Caddy	Naudotojo sąsaja (SPA)
fisingas-django	Django 5.2 + Gunicorn (Python 3.13)	JSON API: autentikacija, testas, vertinimas
fisingas-postgres	PostgreSQL 17	Visa ilgalaikė būseną, įskaitant paveikslėlius
fisingas-cron	BusyBox crond + Docker CLI	Naktinis duomenų saugojimo terminų valymas
fisingas-filebrowser-dropbox	Filebrowser (be autentikacijos)	Bendra failų dėžė personalui
fisingas-filebrowser-slides	Filebrowser (be autentikacijos)	Projektoriaus skaidrių įkėlimas
fisingas-dbgate	DBGate 7.2	Duomenų bazės valdymo sąsaja
fisingas-swagger	Swagger UI	Interaktyvi API dokumentacija

1 lentelė: Docker Compose steko paslaugos.



1 pav.: Paslaugų topologija. Tik Caddy įėjimo taškas publikuoja serverio prievadą; visi kiti konteineriai pasiekiami vien per vidinį Docker tinklą. Dvi Filebrowser kopijos aptarnauja prijungtus `_DATA/dropbox` ir `_DATA/slides` katalogus; Django tą patį skaidrių katalogą skaito projektoriaus režimui. Cron konteineris neparodytas: jis veikia su `network_mode: none` ir Django pasiekia tik per Docker lizdą (`docker exec`).

2.2 Maršrutizavimas

Įėjimo taško Caddy klausosi 80 prievade ir maršrutizuoja pagal kelią, eilės tvarka:

- `/filebrowser/dropbox/*`, `/filebrowser/slides/*`, `/dbgate/*` ir `/swagger*` nukreipiami į atitinkamą įrankį, bet tik po to, kai `forward_auth` pagalbinė užklausa į `fisingas-django:8000 /api/checkauth/admin` patvirtina, kad kreipiasi administratoriaus sesija (4.4 poskyris);
- `/api/*` nukreipiamas į Django be jokio filtro tarpiniame serveryje — kas ką gali kviesti, sprendžia pats Django kiekviename galiniame taške;
- visa kita atitenka naudotojo sąsajos konteineriui, kuriame antra Caddy kopija aptarnauja iš anksto surinktą Vite paketą kaip statinius failus su SPA atsarginiu keliu (`try_files {path} /index.html`), tad gilios nuorodos kaip `/admin/students/42` išsprendžiamos kliento pusėje.

2.3 Saugumo laikysena

Vienodai visame steke taikomos apsaugos priemonės:

- **Vienas įėjimas.** Prievadą publikuoja tik įėjimo taško konteineris. Django, PostgreSQL ir visi įrankiai iš serverio tinklo nepasiekiami.

- **Tos pačios kilmės kontrolė pakraštyje.** Caddy atmeta su 403 kiekvieną užklausą, kurios `Origin` antraštė yra ir neatitinka užklausos serverio vardo. Tai ir yra steko CSRF apsauga — Django CSRF tarpinė programinė įranga sąmoningai neįdiegta (šio pasirinkimo ribos aptariamos 8 skyriuje).
- **Content-Security-Policy.** Griežta CSP (`default-src 'self'`; paveikslėliai papildomai iš `data:` ir `blob:`, ko reikia skaidrių projektoriui), `frame-ancestors 'self'` ir `Strict-Transport-Security` antraštė su vienerių metų galiojimu.
- **Tik skaitomos failų sistemos ir ne root vykdymas.** Įėjimo taško, sąsajos, serverinės dalies ir duomenų bazės konteineriai veikia su `read_only: true` naudotoju `1000:1000`; rašomos vietos apribotos iki išreikštinių tomų ir `tmpfs` prijungimų (Django gauna `tmpfs` ties `/tmp` įkėlimų buferizavimui).
- **Paslaptys ne compose faile.** Django pasirašymo raktas ir DBGate prisijungimo duomenys diegimo skripto sugeneruojami į `.env` (7.2 poskyris); nei `.env`, nei įdiegtas `docker-compose.yml` nėra sekami git.

3 Duomenų modelis

3.1 Esybės

Visa būseną gyvena PostgreSQL vienuolikoje lentelių (2 lentelė). Kiekvienas laiko žymuo yra gimtasis `timestamp_tz` stulpelis, saugantis UTC; API jį pateikia ISO-8601 formatu vietos laiku su aiškiu poslinkiu (`2026-08-26T19:09:07+03:00`, žiemą `+02:00`), o „niekada“ reiškia `null`. Visi palyginimai — skydelio „aktyvūs per paskutines 30 minučių“ langas, saugojimo terminų valymas, sąsajos datų filtrai — atliekami su tikromis datomis, todėl vasaros laiko perėjimai nekenkia, o rodymas teisingas bet kurioje naršyklės laiko juostoje.

Lentelė	Paskirtis
<code>users_systemuser</code>	Administratorių paskyros (el. paštas, bcrypt maiša, įjungimo žymė)
<code>users_student</code>	Studentų paskyros (vardas, atviras prieigos kodas, baigimo žymė)
<code>users_setting</code>	Raktas-reikšmė nustatymai; naudojamas tik <code>PhishingTestSize</code>
<code>phishing_test_questionimage</code>	Laiškų ekrano kopijos dvejetainiais blokais; tik įkėlimas
<code>phishing_test_question</code>	Gyvas, redaguojamas klausimų bankas
<code>phishing_test_questionoption</code>	Papildomi klausimo žymimieji variantai
<code>phishing_test_questionlink</code>	Spaudžiamos nuorodų sritys paveikslėlyje (trupmeninės koordinatės)
<code>phishing_test_answer</code>	Užšaldyta klausimo kopija + studento verdiktas
<code>phishing_test_answerselectedoption</code>	Užšaldyta varianto kopija + studento žymė
<code>phishing_test_testresult</code>	Užšaldytos viso testo sumos, įrašomos baigiant
<code>django_session</code>	Serverio pusės sesijų saugykla

2 lentelė: Duomenų bazės lentelės. „Užšaldytos“ lentelės rašomos išdalijant ar baigiant testą ir vėlesni banko pakeitimai jų niekada nepaliečia.

3.2 Momentinė kopija išdalijant

Kertinis sistemos invariantas: *įvertinimas privalo būti atkuriamas amžinai, kad ir kas nutiktų klausimų bankui*. Tai pasiekama denormalizacija išdalijimo metu: studentui pirmą kartą paprašius savo klausimų, sistema į `Answer` ir `AnswerSelectedOption` eilutes nukopijuoja kiekvieno išdalyto klausimo tekstą, teisingą verdiktą, nuorodą į paveikslėlį ir kiekvieno varianto tekstą su laukiama reikšme. Kopijos į klausimą sąmoningai rodo paprastu sveiku skaičiumi, o ne išoriniu raktu, tad vėlesnis klausimo trynimasis negali kaskadu paliesti išdalyto testo. Vertinimas (5.2 poskyris) skaito išimtinai šias užšaldytas eilutes.

Du unikalumo apribojimai — (`student`, `question_id`) atsakymams ir (`student`, `question_id`, `option_id`) variantams — garantuoja, kad studentas niekada neturės dviejų to paties klausimo kopijų. Pats lygiagretumas tvarkomas lygiu aukščiau: kiekviena testo galinių taškų užklausa pirmiausia atnauja studento „paskutinį kartą matytas“ žymą, o tai PostgreSQL užrakina to studento eilutę, tad dvi vienalaikės pirmos užklaustos nuosekliojaamos — antroji palaukia, tada pamato pirmosios eilutės ir išdalijimą praleidžia. Jei apribojimas vis dėlto suveiktų, užklausa tai užregistruoja žurnale ir aptarnauja nusileidusią kopiją; bet kokia kita vientisumo klaida registruojama ir metama toliau, o ne atsakoma tuščiu testu. Kartu su transakcijomis kiekvienai užklausiai iš dalies išdalytas testas negali egzistuoti.

Studentui baigus įvyksta trečias užšaldymas: suskaičiuotos sumos vieną kartą įrašomos į `TestResult` laikant tą patį eilutės užraktą, tad dar patvirtinamas automatinis išsaugojimas įtraukiamas į užšaldytas sumas — sąrašuose rodomas įvertinimas ir klausimų peržiūra niekada negali nesutapti. Nuo tada kiekvienas sąrašo vaizdas (administratoriaus lentelė, vieša lyderių lentelė) šį studentą aptarnauja iš vienos saugomos eilutės, o ne pervertindamas atsakymus, tad karštieji puslapiai baigusiam studentui dirba pastovų darbą, o gyvas vertinimas lieka tik dar sprendžiantiems.

3.3 Paveikslėlių saugykla

Ekranų kopijos — vienintelis dvejetainis krovinys, joms skirta atskira tik įkėlimams naudojama lentelė. Naujas klausimas sukuriama *įkeliant* paveikslėlį (6.3 poskyris); failas — patikrintas pagal plėtinį (`png|jpg|jpeg|gif`) ir apribotas 5 MB — įrašomas žaliais baitais į PostgreSQL, o aplink jį sukuriamas klausimo eilutė. Aptarnaujant turinio tipas nustatomas iš bloko magiškųjų baitų, o ne pasitikima įkėlimu.

Ir gyvas klausimas, ir užšaldytos atsakymų kopijos į paveikslėlį rodo su `PROTECT`, o paveikslėlių netrina joks kodo kelias: ištrynus klausimą jo ekranų kopija lieka, o `GET /api/phishingpictures/{id}`, klausimui nebeegzistuojant, atsitraukia prie kopijos nuorodos — tad įvertintas testas visada gali iš naujo parodyti savo laiškus.

Spaudžiamos nuorodų sritys (`QuestionLink`) kabo prie *paveikslėlio*, ne prie klausimo — jų koordinatės (saugomos trupmeninėmis tekstinėmis reikšmėmis) prasmingos tik tame konkrečiame rastre, ir dėl šio priskyrimo seni įvertinti testai išlaiko savo užvedamas nuorodas net ištrynus klausimą.

4 Autentikacija ir prieigos kontrolė

4.1 Paskyros ir rolės

Yra lygiai dvi paskyrų rūšys, laikomos atskirose lentelėse, o rolę koduoja pats prisijungimo vardas: administratoriai jungiasi el. pašto adresu, studentai — sugeneruotu vardu. Prisijungimo galinis taškas juos atskiria pagal tai, ar pateiktame varde yra `@` — saugu, nes studentų vardai registruojantis normalizuojami iki abėcėlės `A-Z 0-9 _` (didžiosiomis, visi kiti simboliai išmetami), tad dvi vardų erdvės negali susikirsti.

Administratorių slaptažodžiai maišomi bcrypt (12 raundų); kuriant ar keičiant paskyrą reikalaujama bent 8 simbolių ir ne daugiau kaip 72 baitų — bcrypt įvesties riba; ilgesnis slaptažodis tyliai autentikuotųsi pagal nukirstą pradžią — o el. pašto adresai privalo būti unikalūs ir redaguojant, ne tik kuriant. Studentų prisijungimo duomenys — sistemos sugeneruoti 8 skaitmenų prieigos kodai (10000000–99999999), saugomi ir lyginami *atviru tekstu* — dokumentuotas projektinis sprendimas: kodas yra sistemos išduotas prieigos žetonas, studentui parodomas pagal pareikalavimą, o ne naudotojo pasirinkta paslaptis, kuri galėtų būti naudojama kitur. Nepa-

vykęs nežinomo vardo prisijungimas vis tiek įvykdo bcrypt palyginimą su fiktyvia maiša, tad atsako laikas neišduoda, ar paskyra egzistuoja.

4.2 Sesijos

Django duomenų baze paremtas sesijų karkasas — vienintelė naudojama `django.contrib` dalis. Prisijungimas naujoje sesijos eilutėje išsaugo vieną reikšmę — prisijungimo vardą — ir nustato `HttpOnly` slapuką vardu `session`, galiojantį iki naršyklės uždarymo. Atsijungimas vyksta serverio pusėje: `POST /api/logout` ištrina sesijos eilutę ir išvalo slapuką, tad anksčiau nukopijuota slapuko reikšmė (bendri ar kiosko kompiuteriai) nuo tos akimirkos nebegalioja; prisijungimo puslapis šį kvietimą atlieka kaskart atidarytas — taip veikia nuoroda „Atsijungti“. Kiekvienoje užklausoje sesijos vardas iš naujo išsprendžiamas paskyrų lentelėse, o išjungtos paskyros laikomos neegzistuojančiomis — deaktiviuotas administratorius ar studentas gyvą sesiją praranda su kita savo užklausa.

4.3 Autorizacija vaizduose

Nedidelis nuosavas dekoratorius pakeičia `django.contrib.auth: @login_required` anoniminės užklauskos atmeta su 401 ir prie užklauskos prisega išspręstą paskyrą; rolės tikrinamos jau kiekviename vaizde. Susidaro trys prieigos pakopos: vieši galiniai taškai (prisijungimas, atsijungimas, registracija, lyderių lentelė, skaidrės), autentikuoti (klausimų paveikslėliai ir jų nuorodų sritys) ir administratoriaus — iš kurių du (studento kortelė ir studento atsakymai) papildomai leidžia studentui skaityti *savo paties* įrašą; taip veikia rezultatų puslapis.

4.4 Autorizacija tarpiniame serveryje

Operatoriaus įrankiai (DBGate, abu Filebrowser, Swagger UI) savos autentikacijos neturi. Vietoj to Caddy kiekvienai į juos nukreiptai užklauskai atlieka `forward_auth` pagalbinę užklauską į `GET /api/checkauth/admin`; galinis taškas 200 atsako tik gyvai administratoriaus sesijai, kitaip — 401. Paties DBGate basic-auth duomenys sugeneruojami diegiant, ir Caddy juos įterpia į persiunčiamą užklauską, tad naršyklė jų niekada nemato. Tas pats administratoriaus tikrinimo taškas kartu yra „paskutinį kartą matytas“ pulsas: kiekvienas sėkmingas kvietimas atnaujiną paskyros paskutinio prisijungimo žymą, kuri maitina skydelio aktyvumo vaizdą.

5 Testo gyvavimo ciklas

5.1 Vienas studentas nuo pradžios iki pabaigos

Studentas pereina šias stadijas (2 pav.):

1. **Registracija.** `POST /api/student/register` su norimu slapypavardžiu. Vardas paverčiamas didžiosiomis ir apkarpos iki A-Z 0-9 _; jei laisvas, paskyra sukuriamą su atsitiktiniu 8 skaitmenų prieigos kodu, atsake grąžinamą lygiai vieną kartą. Sąsaja abu parodo su įspėjimu „užsirašykite“.
2. **Prisijungimas.** `POST /api/login` su vardu ir kodu nustato sesijos slapuką. Atsako kūnas — arba pažodinė eilutė "OK", arba rodymui paruoštas lietuviškas klaidos pranešimas.
3. **Išdalijimas.** Pirmasis `GET /api/student/questions` paleidžia išdalijimą: testo dydis skaitomas iš `PhishingTestSize` nustatymo (atsarginė kodo reikšmė 30, administratoriui siūlomi variantai 9/12/15/21/30), iš *įjungty* klausimų ištraukiama tolygiai atsitiktinė tokio dydžio imtis (apribota banko dydžiu) ir masiškai įrašoma 3.2 poskyryje aprašyta užšaldyta kopija. Vėlesni kvietimai grąžina jau išdalytą rinkinį ta pačia tvarka: tvarka

kiekvienam studentui atsitiktinė ir fiksuojama išdalijant, tad navigacijos skydelio numeracija išgyvena puslapio perkrovimą. Klausimų aibė skaitoma vieną kartą ir imtis dydinama pagal tą patį skaitymą, o sugadinta nustatymo reikšmė atsitraukia į numatytąją kaip ir trūkstanta, tad pirma užklausa negali sugriūti dėl tuo pat metu redaguojamo banko ar ranka pakeisto nustatymo.

4. **Atsakinėjimas.** Sąsaja po kiekvieno paspaudimo siunčia *visą* atsakymų būseną į `POST /api/student/questions` — verdiktus (0 = tikras, 1 = fišingas) į atsakymų eilutes, žymių būsenas į variantų eilutes. Kiekvienas atnaujinimas apribotas sesijos studento identifikatoriumi, tad suklustotas krovinsys negali rašyti į kito studento eilutes. Visas krovinsys patikrinamas prieš ką nors įrašant, tad netaisyklingas išsaugojimas atmetamas kaip visuma ir niekada nepalieka pusiau įrašytos būsenos; sąsaja išsaugojimus siunčia po vieną, paspaudimus, padarytus dar vykstant išsaugojimui, sujungdama į vieną paskesnę, tad senesnė būsena niekada negali perrašyti naujesnės. Saugojimas idempotentinis ir kartotinas; studentas gali užverti naršyklę ir tęsti prisijungęs su vardu ir kodu.
5. **Baigimas.** `GET /api/student/finish` užrakina studento eilutę, įvertina užšaldytas eilutes, įrašo `TestResult` sumas ir nustato baigimo žymę; jau baigęs studentas paliekamas nepaliestas, tad dvigubas paspaudimas nekenksmingas. Sąsaja prieš pereidama prie šio kvietimo palaukia, kol bus patvirtintas dar vykstantis automatinis išsaugojimas, tad paskutinis paspaudimas visada yra užšaldyto įvertinimo dalis. Žingsnis negrįžtamas: nuo šiol klausimų galinis taškas šiam studentui grąžina tuščią objektą, o maršruto sargas nukreipia jį į rezultatų puslapį.
6. **Rezultatai.** Studentas savo santrauką ir klausimų peržiūrą — įskaitant teisingus atsakymus — skaito per tuos pačius galinius taškus, kuriais administratorius žiūri bet kurį studentą; vieša lyderių lentelė rodo jo įvertinimą.

5.2 Vertinimas

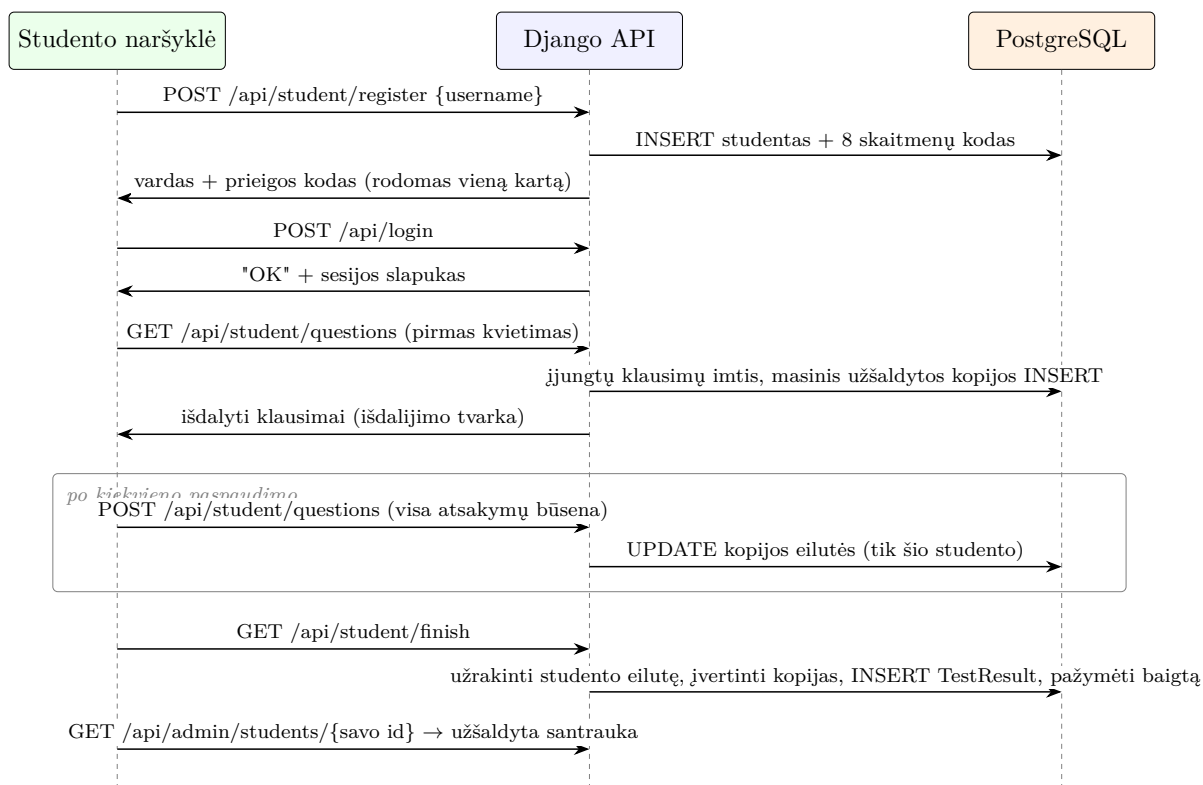
Vertinimas skaito tik užšaldytą kopiją. Išdalytam klausimui q tegul $v_q \in \{0, 1\}$ — teisingas verdiktas, a_q — studento verdiktas, n_q — papildomų variantų skaičius, o c_q — variantų, kurių žymės būseną sutampa su laukiama reikšme, skaičius (nepaliesta žymė laikoma nepažymėta). Klausimas vertas daugiausia vieno taško, o papildomi variantai aplink jį veikia kaip 0,1 taško priedo/nuoskaitos juosta:

$$p_q = \begin{cases} 0 & \text{jei } a_q \neq v_q, \\ 1 - 0,1 n_q + 0,1 c_q & \text{jei } a_q = v_q. \end{cases}$$

Tai yra, teisingas verdiktas su visais teisingais variantais duoda lygiai 1; kiekvienas praleistas variantas kainuoja 0,1. Jei pagrindinis verdiktas klaidingas, klausimas vertinamas 0 nepriklausomai nuo variantų. Testui iš N išdalytų klausimų galutinis įvertinimas lietuviškoje dešimtbalėje skalėje yra

$$G = \text{round}\left(\frac{10}{N} \sum_{q=1}^N p_q, 2\right),$$

pateikiamas fiksuota dviejų skaitmenų po kablelio eilutė (pvz. "8.50"). Santrauka papildomai nurodo, kiek klausimų identifikuota teisingai (vien verdiktas), kiek *visiškai* teisingi (verdiktas ir visi variantai), variantų sumas ir visiškai teisingų dalį nukirsto sveiko skaičiaus procentais. Variantų sumos skaičiuojamos tik teisingai identifikuotuose klausimuose — lygiai tie variantai,



2 pav.: Vieno studento pranešimų seka. Išdalyjimas per pirmą klausimų užklausą užšaldo kopiją, su kuria dirba visas vėlesnis atsakinėjimas ir vertinimas; baigimo kvietimas sumas užšaldo antrą kartą, po kurio sąrašų vaizdai šio studento nebepervertina.

kuriuos vertina formulė — tad nieko neatsakęs studentas mato 0/0, o ne kreditą už nepalistas žymes, kurios šiaip laukė „nepažymėta“. N skaičiuoja *išdalytus* klausimus, tad neatsakyti klausimai į įvertinimą įeina lygiai kaip klaidingi verdiktai. Naujai pridėtas variantas laukia „nepažymėta“, kol administratorius nenurodo kitaip; niekada nenustatyta laukiama reikšmė (galima tik už šią taisyklę senesnėse eilutėse) negali būti atitikta.

Dar sprendžiantiems studentams šie rodikliai skaičiuojami gyvai iš kopijos eilučių dviem aibinėmis užklausomis; baigus jie užšaldomi į `TestResult` ir nuo tada aptarnaujami iš ten visam laikui (3.2 poskyris).

6 Naudotojo sąsaja

6.1 Technologija

Naudotojo sąsaja — React 19 vieno puslapio aplikacija, renkama su Vite 7, valdikliams ir duomenų lentelėms naudojanti MUI 7, išdėstymui — Tailwind 4, o kliento pusės maršrutams — `react-router` 7. Nėra nei būsenos valdymo bibliotekos, nei kliento podėlio: vienas konteksto tiekėjas kartą per pilną puslapio įkėlimą patikrina `GET /api/checkauth`, o maršrutų sargai nukreipia pagal rolę; serverio būseną traukia nedidelis bendras kablys, palaikantis intervalinį klausinėjimą ir kiekvieną 401 paverčiantis kietu nukreipimu į prisijungimo puslapį. Visas naudotojui matomas tekstas — lietuviškas (įrašytas kode, i18n sluoksnio nėra); sąsaja spalvinta fakulteto bordo (`#7B003F`) su savarankiškai talpinamu Inter kintamu šriftu, kurio latin-extended poaibis dengia lietuviškus diakritikus. Gamybinis atvaizdas renkamas dviem etapais — Node etapas įvykdo `vite build`, rezultatas nukopijuojamas į Caddy atvaizdą — tad platinamame konteineryje tėra statiniai failai ir SPA atsarginio kelio žiniatinklio serveris, be Node vykdymo

aplinkos.

Vietoj patvirtinimo dialogų visur naudojamas firminis sąveikos šablonas: destruktivūs ar negrįžtami veiksmai (baigti testą, trinti klausimą, studentą, administratorių) — *palaikyk, kad patvirtintum* mygtukai, kuriuos reikia laikyti 1,5 sekundės, kol prisipildo progreso žiedas, ir pele, ir lietimui.

6.2 Studento patirtis

Prisijungimo puslapis pagal nutylėjimą rodo *registraciją* — nauji studentai yra pagrindinė auditorija. Užsiregistravus vardas ir prieigos kodas parodomi vieną kartą, lygiaplotėmis kortelėmis po gintarinį įspėjimą „užsirašykite“, su vienu mygtuku, kuris prijungia ir pradeda testą.

Testo puslapis rodo po vieną laiško ekrano kopiją, pusės ekrano aukščio, su spustelėjimu priartinama iki viso ekrano peržyklės (uždaroma fono paspaudimu arba **Escape**). Virš ekrano kopijos platforma imituoja tą vieną elgseną, kurios statinis paveikslėlis duoti negali: *nuorodų peržiūra*. Nematomi taškinių sričių stačiakampiai — administratoriaus apibrėžti trupmeninėmis koordinatėmis — seka atvaizduojamą paveikslėlio dydį, o užvedus ant srities virš žymeklio parodomas juodas debesėlis su tikruoju nuorodos URL, mėgdžiojantis tikro pašto kliento būsenos juostos peržiūrą. Studentams sritys sąmoningai piešiamos visiškai skaidrios, tad jas rasti galima tik iš tiesų tyrinėjant laišką.

Po paveikslėliu — verdikto mygtukai *Tikras* ir *Fišingas* bei klausimo papildomos žymės. Kiekvienas paspaudimas automatiškai išsaugo visą atsakymų būseną serveryje (vienu metu vyksta vienas išsaugojimas, vėlesni paspaudimai sujungiami), o nesėkmę praneša iššokantis pranešimas, tad nutrūkęs ryšys ar užvertas nešiojamasis nieko nekainuoja: prisijungus iš naujo tęsiama lygiai ten, kur sustota. Progreso skydelis rodo atsakytų skaitiklį, progreso juostą ir šuolių tinklėlį su mygtuku kiekvienam klausimui (užpildytas — atsakyta, apvestas — dabartinis). Baigimo mygtukas — 1,5 sekundės palaikymas, kuris prieš paliekant puslapį palaukia paskutinio automatinio išsaugojimo patvirtinimo — jei jo pristatyti nepavyksta, studentui apie tai pranešama ir jis lieka teste, užuot baigęs be tų atsakymų. Rezultatų puslapis tada rodo įvertinimo antraštę, tris santykių korteles (visiškai teisingi, teisingai identifikuoti, teisingi variantai), studento vardą su prieigos kodu ir tą patį klausimų peržiūros komponentą, kurį mato administratorius — atsakymų raktų stulpelis neutraliai pilkas, studento pasirinkimai prieš jį spalvinami žaliai arba raudonai.

6.3 Administratoriaus patirtis

Administratoriai gauna sutraukiamą šoninės juostos apvalkalą (būsena išsaugoma `localStorage`) su aplikacijos puslapiais ir išorinėmis nuorodomis į saugomus įrankius (skaidrių tvarkyklė, failų dėžė, Swagger, DBGate).

Skydelis. Klausinėja API kas 2 sekundes ir rodo studentų skaičių, įjungtų/visų klausimų skaičius ir gyvą „šiuo metu sprendžia“ sąrašą — po progreso juostą kiekvienam per paskutines 30 minučių aktyviam studentui, baigus žaliuojančią. Testo dydžio parinkiklis (9/12/15/21/30) sėdi tiesiai klausimų valdiklyje ir išsaugo iškart pakeitus; atskiros nustatymų puslapio nėra.

Klausimų bankas. Klausimas sukuriamas įkeliant ekrano kopiją (tempimas, peržiūra, 5 MB riba); naujas klausimas tada pasirodo kaip pildytina kortelė. Kiekviena kortelė redaguojama vietoje su uždelstu automatinio saugojimu: praėjus 500 ms po paskutinio klavišo kortelė išsisaugo, o būsenos ženklelis pereina *Saugoma.../Išsaugota* — redaktoriuje niekur nėra saugojimo mygtuko. Variantų pridėjimas ar trynimasis ir klausimo trynimasis saugomi iškart (jie kuria ar šalina eilutes); trynimai — palaikymo patvirtinimu. Kortelėje — fišingo verdikto žymė, nepriwalomas papildomo nurodymo tekstas, variantų sąrašas su teisingomis reikšmėmis ir įjungimo

jungiklis — dalijami tik įjungti klausimai, o kai įjungtų mažiau nei nustatytas testo dydis, rodoma įspėjimo juosta. Filtro juosta ieško pagal tekstą, varianto tekstą ar ID; atfiltruotos kortelės slepiamos, o ne išmontuojamos, tad neišsaugoti pakeitimai išgyvena filtro keitimą.

Taškinių sričių redaktorius atsiveria per visą ekraną virš kortelės: nuorodų sritys braižomos, tempiamos ir keičiamo dydžio tiesiai ant ekrano kopijos (**react-rnd**), kiekviena su savo URL, ir išsaugomos trupmeninėmis koordinatėmis. Tas pats redaktorius pasiekiamas tiesiogiai adresu `/admin/questions/{id}`.

Studentai. Duomenų lentelė (5 sekundžių klausinėjimas, greitas filtras, numatytasis „per paskutinį mėnesį“ jungiklis) išvardija kiekvieną studentą su progresu, įvertinimu ir laiko žymomis, rikiuojamomis kaip skaičiai ir datos, o ne tekstas. Kortelės puslapis rodo prisijungimo duomenų kortelę — įskaitant prieigos kodą, kad dėstytojas galėtų išgelbėti jį pametęsį studentą — santraukos korteles ir dvi kirptis: klausimų peržiūrą ir sutrauktą lentelę su ekrano kopijų miniatiūromis. Studentus galima trinti (palaikymo patvirtinimu); kaskadas pašalina jų išdalytą testą ir įvertinimus, bet paveikslėlių niekada neliečia.

Administratoriai. Lentelė ir vienas modalinis langas kūrimui/redagavimui/trynimui per vieną galinį tašką; tuščias `id` reiškia įterpimą, tuščias slaptažodis redaguojant — „palikti dabartinį“. Slaptažodis turi būti nuo 8 simbolių iki 72 baitų, el. pašto adresai — unikalūs, o administratorius negali nei išjungti, nei ištrinti savo paties paskyros — tad bent viena veikianti administratoriaus paskyra išlieka visada.

6.4 Viešieji rodiniai

Du neautentikuojami puslapiai skirti projektoriui:

- **Lyderių lentelė** (`/leaderboard`) — rikiuoja studentus pagal įvertinimą su aukso/sidabro/bronzos ženkleliais, dar sprendžiantiems rodo mėlynas progreso juostas, baigusiems — bordo įvertinimo juostas, atsinaujina kas 5 sekundes su matoma atskaita ir pagal nutylėjimą filtruoja iki per paskutinę parą matytų studentų.
- **Skaidrių kioskas** (`/slides`) — kaitalioja lyderių lentelę (įterptą iframe už paspaudimus blokuojančio sluoksnio, 6 sekundės) ir atsitiktinę skaidrę iš skaidrių katalogo (20 sekundžių). Kita skaidrė parsiuočiama ir dekoduojama kaip blob *dar rodant lyderių lentelę*, tad keitimas visada akimirksninis; skaidrių failus administratoriai tvarko per skaidrių Filebrowser.

7 Diegimas ir eksploatavimas

7.1 Būtinės sąlygos

Linux serveris su Docker ir Docker Compose bei laisvas 80 prievadas. Kitų priklausomybių nėra: atvaizdai traukiami iš Docker Hub (`vuknf/fisingas-*`), o kiekviena paslauga compose faile turi ir užkomentuotą `build`: variantą rinkimui vietoje.

7.2 Steko paleidimas

Vienintelis rankinis žingsnis — sukurti compose failą iš pavyzdžio; visa kita — vienas idempotentinis skriptas:

```
cp docker-compose.yml.sample docker-compose.yml
./runUpdateThisStack.sh
```

Skriptas sukuria `_DATA` katalogus (`postgres`, `slides`, `dropbox`) ir sutvarko jų nuosavybę, patikrina, ar įdiegtas Docker, sugeneruoja `DJANGO_SECRET_KEY` ir `DBGate` prisijungimo duomenis į `.env` (vieną kartą), sukuria Docker tinklą, (iš naujo) paleidžia steką, kartojimo cikle pritaiko duomenų bazės migracijas (pirmas PostgreSQL paleidimas trunka kelias sekundes) ir — tik tuščioje duomenų bazėje — sukuria pirmą administratoriaus paskyrą bei išspausdina jos duomenis:

```
El. pastas:    admin@admin.com
Slaptazodis:  admin
```

Ši slaptažodį būtina nedelsiant pakeisti administratorių puslapyje. Paslauga tada pasiekama serverio 80 prievade: `/` — aplikacija, `/swagger` — API dokumentacija, `/dbgate` — duomenų bazės sąsaja (visi trys įrankių keliai saugomi administratoriaus sesija).

Kiekvieną užklausą Unicorn registruoja konteinerio standartinėje išvestyje (`docker logs fisingas-django`): Caddy persiuntą kliento adresą, užklauskos eilutę, būsenos kodą, atsako dydį ir trukmę.

7.3 Duomenų saugojimo terminai

Cron konteineris — BusyBox `crond` su Docker CLI, be tinklo ir su į atvaizdą įkepta `crontab` — vykdo vieną darbą: kas naktį 05:00 (valanda, kuri egzistuoja kiekvieną metų dieną; 03:00 yra būtent ta, kurią vasaros laikas pavasarį praleidžia, o rudenį pakartoja) Django konteinyje paleidžia valdymo komandą `delete_old_students`. Komanda ištrina studentų paskyras, kurių paskutinis prisijungimas ir registracija abu senesni nei 180 dienų; kaskadas pašalina jų išdalytus testus ir įvertinimus, o klausimų paveikslėliai niekada neliečiami. Šviežios, nė karto neprisijungusios paskyros išlieka. Visa kita saugoma neribotai: klausimų bankas, paveikslėliai, administratorių paskyros ir `_DATA` katalogai serveryje.

7.4 Konfigūracijos žinynas

Visi nustatymai — aplinkos kintamieji, nustatomi `docker-compose.yml` arba sugeneruojami į `.env` (3 lentelė). Vienintelis aplikacijos lygmens nustatymas — testo dydis — gyvena duomenų bazėje ir redaguojamas iš skydelio.

Kintamasis	Numatytoji	Reikšmė
<code>DJANGO_SECRET_KEY</code>	generuojamas į <code>.env</code>	Pasirašo sesijos kodo numatyti
<code>DATABASE_URL</code>	<code>postgres://fisingas:...@fisingas-postgres:5432/fisingas</code>	PostgreSQL duomenų
<code>SLIDES_DIRECTORY</code>	<code>/slides</code>	Skaidrių katalogas
<code>DJANGO_DEBUG</code>	nenustatytas (<code>False</code>)	Perjungia konteinerio serverį su debug režimu
<code>POSTGRES_DB</code> / <code>_USER</code> / <code>_PASSWORD</code>	<code>fisingas</code>	Duomenų bazės pavadinimas, vartotojas, slaptažodis
<code>DBGATE_PASSWORD</code>	generuojamas į <code>.env</code>	DBGate bazės slaptažodis
<code>DBGATE_AUTH_HEADER</code>	generuojamas į <code>.env</code>	Iš anksto užrašytas slaptažodis, kurį Caddy

3 lentelė: Konfigūracijos žinynas. Nustatymas `PhishingTestSize` (klausimų skaičius teste; kodo atsarginė reikšmė 30, skydelio variantai 9–30) saugomas duomenų bazėje, ne aplinkoje.

7.5 Kūrimo režimas

Kiekvienas paslaugos blokas `compose` faile turi užkomentuotas `Dev` eilutes: jas įjungus išeities kodas prijungiamas į konteinerius ir vietoje gamybinių serverių paleidžiamas Vite kūrimo ser-

veris (su karštu perkrovimu) ir Django `runserver` (per `DJANGO_DEBUG=True`). Serverinės dalies atvaizdas penkias savo priklausomybes prisega tiesiai Dockerfile — `requirements.txt` nėra — o lydintys `pushDockerhub.sh` skriptai kiekvieną atvaizdą surenka ir publikuoja su `latest` ir datos žymomis.

7.6 Regresiniai testai

`django/runTests.sh` surenka serverinės dalies atvaizdą ir laikiname konteineryje paleidžia regresinių testų rinkinį — daugiau nei tris šimtus testų — su atmintyje laikoma SQLite duomenų baze, tad veikiančio steko nereikia. Rinkinys dengia kiekvieną galinį tašką, vertinimo aritmetiką, momentinių kopijų ir užšaldymo discipliną, saugojimo terminų valymą ir laiko žymų politiką; kontraktų sluoksniš gyvus atsakus tikrina pagal `swagger/swagger.yaml`, tad API dokumentacija negali nepastebimai nutolti nuo kodo. Žinomi, dar nepataisyti defektai prisegti kaip *laukiamos nesėkmės*: kiekvienas turi testą, tvirtinantį teisingą elgseną, tad rinkinys lieka žalias, o defektas — dokumentuotas ir vykdomas, o pataisymas testą apverčia, užuot tyliai praėjęs.

8 Apribojimai ir ateities darbai

Žvilgsnis iš išeities kodo lygmens aiškiai parodo ir sistemos ribas. Platforma sukurta prižiūrimai auditorijai patikimame tinkle, ir keli sprendimai prasmingi tik tokioje aplinkoje:

- **Grynas HTTP.** Įėjimo taškas klausosi 80 prievade be TLS; gamybiniuose diegimuose TLS turėtų būti terminuojamas prieš įėjimo tašką. Sesijos slapukas neturi `Secure` žymės, tad transporto saugumas — vien perimetro darbas.
- **Nėra užklausų ribojimo.** Prisijungimas ir registracija neautentikuoti ir neribojami. Studentų prieigos kodai — 8 skaitmenų skaičiai, tad brutalaus jėgos bandymas internetu tėra skaičių žaidimas, kurio pati platforma nestabdo; taip pat niekas neriboja masinės registracijos. Prieigos kodai ir klausimų imtis traukiami iš numatytojo Python PRNG, ne kriptografinio.
- **CSRF apsauga — tik kilmės tikrinimas.** Django CSRF tarpinė įranga išjungta pagal dizainą; Caddy atmeta svetimas kilmes, bet užklausos be `Origin` antraštės praeina, o baigimo galinis taškas yra būseną keičiantis GET — sukurpta tarpšvėtaininė navigacija galėtų negrįžtamai užbaigti prisijungusio studento testą.
- **Atsakymų raktas užsispyrusiam studentui pasiekiamas dar sprendžiant.** Savo įrašo skaitymo taisyklė rezultatų galiniuose taškuose neturi baigtumo tikrinimo, tad API skaitantis studentas gali parsisiųsti savo įvertintus atsakymus — įskaitant teisingus verdiktus — dar atsakinėdamas. Taip pat bet kuris autentiкуotas studentas pagal ID gali parsisiųsti bet kurį banko paveikslėlį ir jo nuorodų sritis, ne tik jam išdalytus. Sąžiningo naudojimo tai neveikia; neprižiūrimam aukštos svarbos naudojimui šiems taškams reikėtų baigtumo užkardo.
- **Lyderių lentelė vieša pagal dizainą** ir kiekvienam, kas pasiekia serverį, atskleidžia vardus, įvertinimus ir paskutinio matymo laikus. Studentai renkasi slapyvardžius patys — tai sušvelnina, bet nepanaikina atskleidimo.
- **Nėra audito sekos.** Administratoriaus veiksmai ir prisijungimai niekur neregistruojami, išskyrus Unicorn užklausų žurnalą, o ištrinto studento ar klausimo atkurti neįmanoma. Paveikslėliai gyvena duomenų bazės blokais — paprasta ir nuoseklu, bet duomenų bazė tampa dydžio butelio kakleliu.

- **Nėra laiko limitų ir pradžios žymos.** Niekas neužfiksuoja, kada testas buvo išdalytas, tad sprendimo trukmės negalima nei riboti, nei vėliau atkurti; saugomas tik baigimo laikas.
- **Viena kalba.** Visas sąsajos tekstas — kode įrašyta lietuvių kalba; internacionalizacijai sąsajoje reikėtų il8n sluoksnio.

9 Išvados

Fisingas parodo, kad tikslinga auditorijos vertinimo platforma nereikalauja sunkiasvorės infrastruktūros. Vienas Caddy įėjimas, plonas Django API, PostgreSQL ir React SPA — devyni konteineriai, vienas skriptas — užtikrina pilną ciklą: savitarnos registraciją su generuojamais prieigos kodais, vizualų testą, kurio imituojamos nuorodų peržiūros moko paties vertingiausio fišingo atpažinimo įpročio, gyvą progresą dėstytojo skydelyje ir projektuojamą lyderių lentelę, pratimą paverčiančią žaidimu. Lemiamas inžinerinis sprendimas — momentinių kopijų disciplina: išdalijimas užšaldo klausimų turinį studento eilutėse, o baigimas užšaldo sumas, tad klausimų bankas gali laisvai kisti — kurso viduryje, net testo viduryje — niekada nesutrikdydamas jau suteikto įvertinimo. Visas stekas atkuriamas iš vieno compose failo ir prižiūrimas bet kurio, kam patogus Docker.